Invisible Identity: Secure Chat with Blockchain-Based Authentication and Zero-Knowledge Proofs

Problem Statement

In today's digital world, **private and secure communication** is becoming increasingly important—especially for users concerned with privacy, such as journalists, activists, or whistleblowers. However, most popular messaging applications like WhatsApp, Telegram, or Signal still require users to **register using personal details** (such as phone numbers or email addresses). This introduces the risk of **identity exposure**, even if the messages themselves are encrypted.

Additionally, many platforms collect **user metadata**, such as IP addresses, device information, and login patterns. This metadata can be analyzed to track users' activity or associate identities with their messages—defeating the purpose of using "secure" apps.

Even blockchain-based solutions like Ethereum provide only **pseudonymity**, not full anonymity. While users are identified by wallet addresses rather than names, all transaction data on Ethereum is **publicly accessible**. Anyone can analyze the transaction history of an address, and using techniques like **graph analysis**, it's possible to **deanonymize** users over time. This makes blockchains unsuitable for truly private communication—**unless enhanced with extra privacy tools**.

To overcome these limitations, we propose a **new design for an anonymous one-to-one chat system**. In this system:

- **User authentication** is done using **blockchain-based identity commitments**, but no actual personal data (like names, emails, or even wallet addresses) is ever shared.
- The blockchain is used **only to verify identity ownership**, not to store messages.
- All **messages are end-to-end encrypted**, meaning only the sender and recipient can read them.
- Messages are **ephemeral**, meaning they automatically disappear after a short time and are not saved anywhere—not even on the server.

We use **Zero-Knowledge Proofs (ZKPs)** as a key privacy technology. ZKPs allow a user to **prove something is true (like being a valid member of a chat group)** without revealing any additional information. This makes them perfect for **anonymous authentication**—users can prove they are authorized to chat without revealing who they are.

A real-world example of this approach is **DarkFi's DarkIRC**, which allows users to chat without linking their messages to any identifiable account. Messages are encrypted and routed through decentralized infrastructure, ensuring that **identities remain unlinkable and data remains confidential**.

Our project draws inspiration from such systems and aims to build a **simple, web-based chat platform** that delivers:

- **Confidentiality** (through encryption),
- **Anonymity** (through ZKPs and no metadata),
- And **Non-repudiation** (through message signing and cryptographic proof of origin).

In short, the system combines **blockchain, encryption, and zero-knowledge proofs** to allow people to communicate **without sacrificing their privacy**—something that traditional apps cannot fully guarantee.

Objectives

- **Anonymous Authentication:**
  The system allows users to **prove they are authorized to use the platform without revealing their identity**. This is done using **Zero-Knowledge Proofs (ZKPs)**, which enable a user to demonstrate membership in a valid group (such as registered users) **without disclosing who they are**. Technologies like **Semaphore** facilitate this by generating identity commitments and proving membership through ZKPs. This ensures **privacy-preserving authentication** that does not require usernames, passwords, or any personal information.

- **End-to-End Encryption (E2EE):**
  Every message sent between two users is **encrypted on the sender's device and decrypted only on the recipient's device**. This ensures that no intermediary—whether it's the server, network provider, or even a hacker—can read the message. The system uses secure cryptographic standards (e.g., **AES, ECDH**) implemented through libraries like **Web Crypto API** or **PyNaCl**. This guarantees **confidentiality** even during transmission.

- **Ephemeral Messaging:**
  Messages are **temporary** and automatically **self-destruct** after a defined time period (e.g., 30 seconds or 1 minute). This feature prevents the creation of long-term logs or chat history, ensuring that **no lasting trace of communication** exists. It enhances privacy by protecting users even if their devices or browsers are compromised later.

- **Decentralized Identity with Blockchain:**
  Instead of creating user accounts with personal information, the system uses **cryptographic identity commitments** stored on the **Polygon Mumbai testnet**, a blockchain compatible with Ethereum. These commitments serve as anonymous identities that are **tamper-proof and verifiable**. **MetaMask** is used to manage the blockchain wallet and handle identity-related transactions, enabling users to authenticate through their wallet without disclosing their actual Ethereum address or identity.

- **Non-Repudiation:**
  To ensure that users **cannot deny sending a message**, each message is **digitally signed** using the sender's private key (linked to their identity commitment). This

signature proves that the message originated from a specific anonymous identity, allowing **verifiable accountability** without compromising privacy.

- **Beginner-Friendly Tools and Stack:**
  The project uses tools and frameworks that are **well-documented and widely supported**, making it suitable for student developers. These include:
  - **React** for the frontend interface.
  - **Flask or Node.js** for the backend (you'll use Flask).
  - **MetaMask** for blockchain wallet integration.
  - **Solidity** for writing smart contracts.
  - **Semaphore and SnarkJS or Python equivalents** for handling ZKPs.
    The choice of tools ensures the system is both powerful and accessible for academic development.
- **Documentation & Deliverables:**
  A complete Final Year Project (FYP) submission will be produced, which includes:
  - A **comprehensive project report** covering the background, design, implementation, testing, and conclusion.
  - A **technical implementation guide** that documents the development steps, technologies used, and integration details.
  - A **development timeline** (e.g., Gantt chart) outlining the phases and progress checkpoints of the project.

Scope

- The project is a **web-based, one-to-one anonymous chat application** focused on **secure, encrypted, and private communication** between two users.
- Users **do not register with names, emails, or phone numbers**. Instead, they authenticate using **blockchain-based identity commitments** and **zero-knowledge proofs (ZKPs)**.
- The **Polygon Mumbai testnet** is used to store **cryptographic identity commitments**, while the actual chat messages are **not stored on-chain**.
- The chat is **ephemeral**: messages are automatically deleted from the interface after a short period and are **not saved** to any database.
- The backend is developed using **Python (Flask)**, which handles real-time communication (via WebSocket), message routing, and optional ZKP processing.
- The frontend uses **React**, which connects to the blockchain via **MetaMask** and manages user interaction and encryption in the browser.
- The chat uses **end-to-end encryption (E2EE)** and ensures that **no personal data or metadata (IP, browser info)** is collected or stored.

Technology Stack & Tools

🔧 **Frontend**

- **React JS**: Responsive user interface and chat components.
- **MetaMask**: Ethereum wallet used for blockchain interaction and identity management.
- **Ethers.js**: JavaScript library for communicating with the blockchain from the frontend.
- **Web Crypto API / TweetNaCl.js**: For browser-based encryption and message signing.

🧠 **Backend (Python-Based)**

- **Flask**: Lightweight Python web framework for serving the chat API and WebSocket communication.
- **Flask-SocketIO**: For real-time one-to-one messaging using WebSocket.
- **web3.py**: Interact with smart contracts on the Polygon Mumbai testnet (e.g., reading and writing identity commitments).
- **PyNaCl / cryptography**: Python libraries for secure encryption and signing (end-to-end encryption and non-repudiation).
- **PySNARK / zksk**: For generating and verifying Zero-Knowledge Proofs (ZKPs) in Python.

⚙️ **Smart Contracts**

- **Solidity**: Programming language used to write smart contracts for identity management and ZKP verification.
- **Hardhat**: Development environment for compiling, testing, and deploying smart contracts.
- **Polygon Mumbai Testnet**: Blockchain test network for storing identity commitments and verifying ZKPs without cost.

📄 **Development & Deployment Tools**

- **Node.js** (for Hardhat and frontend build scripts)
- **Vite / Create-React-App**: To scaffold and build the React frontend.
- **Ganache (optional)**: Local Ethereum testnet for contract testing.
- **Alchemy or Infura**: Blockchain RPC provider for deploying and interacting with Polygon Mumbai.
- **MetaMask Faucet**: For obtaining free testnet MATIC tokens to test smart contract transactions.

Security and Privacy Aspects

🔐 **Confidentiality**

- All messages are **end-to-end encrypted** using secure cryptographic standards (ECDH + AES-GCM).
- Only the intended recipient can decrypt a message; the server only sees ciphertext.
- The system ensures **message confidentiality**, even in the presence of a compromised server.

🕵️ **Anonymity & Privacy**

- No usernames, emails, or wallet addresses are exposed.
- Users are identified only via **Merkle-based identity commitments** stored on-chain.
- Authentication is performed via **Zero-Knowledge Proofs**, allowing users to prove group membership without revealing any personal data.
- **No metadata** such as IP addresses or device info is collected.

⏳ **Ephemeral Messaging**

- Messages are **not stored** in any database.
- Each message is **automatically deleted** after a short period (e.g., 30 seconds), both from the frontend and backend memory.
- No chat logs or history is kept anywhere, protecting users against later leaks or breaches.

📜 **Non-Repudiation**

- Each message is digitally signed using the sender's private key (e.g., from PyNaCl or identity commitment key).
- This ensures that the message's origin can be **cryptographically verified**, preventing the sender from denying it later.
- Combined with on-chain identity verification, this enforces accountability **without sacrificing anonymity**.

🔎 **Integrity & Authenticity**

- The system verifies that every received message was **not modified** and indeed came from the claimed sender using **digital signatures**.
- This prevents impersonation or tampering with the message content.